

SACC

2020 中国系统架构师大会

SYSTEM ARCHITECT CONFERENCE CHINA 2020

架构融合 云化共建

LIVE 2020年10月22日 - 24日网络直播

架构融合
云化共建

腾讯广告在线数据系统核心能力与架构演进

林兆祥 腾讯广告

目录

1 Marvel存储系统简介

1.1 Marvel简介 / 1.2 核心业务场景 / 1.3 核心能力

2 Marvel批量发布架构演进

2.1 基于内存KV架构：满足实时响应需求 / 2.2 HBase-Extended Marvel架构：解决成本问题

2.3 HBase-Free Marvel架构：解决发布速度问题 / 2.4 HBase-Free Marvel架构分阶段实施

3 Marvel流式发布架构演进

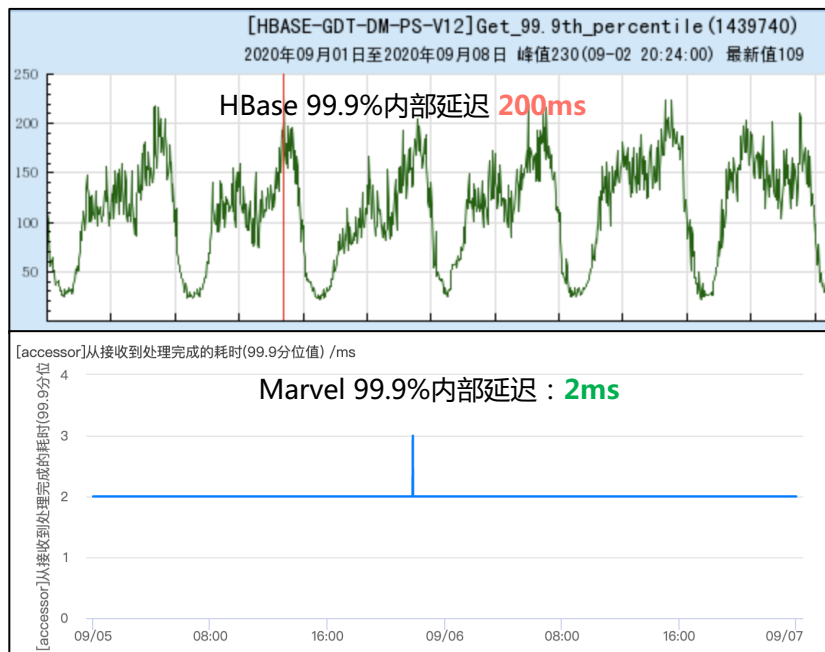
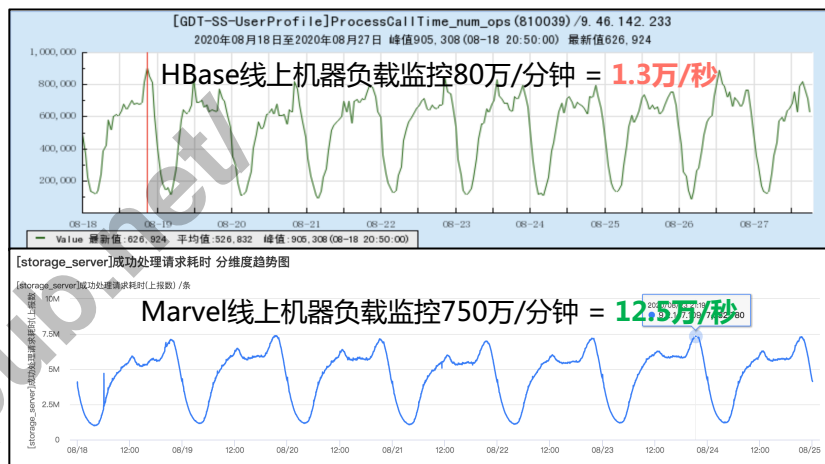
3.1 HBase-Extended架构：满足流式数据发布需求 / 3.2 HBase-Free架构：解决数据一致性问题

3.3 HBase-Free架构分阶段实施

4 总结

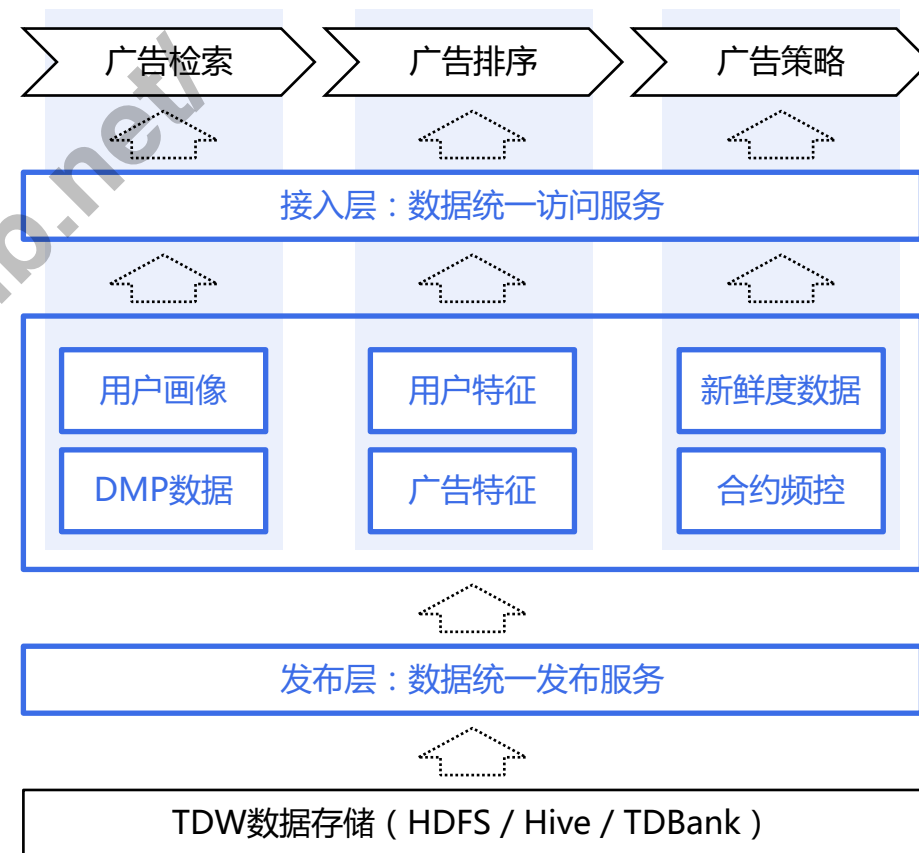
Marvel简介

- Marvel是腾讯广告自研的KV存储系统，借鉴了HBase的原理和架构
- 跟HBase相比，Marvel做了大量的工程优化，在我们业务场景下，各方面性能均远超HBase
 - 单机性能比HBase高10倍以上
 - 99.9%响应速度比HBase高几十倍
 - 日发布数据上百批次，1T数据发布时延迟低于1小时



Marvel定位：广告业务数据存储系统，为广告检索 / 排序 / 策略提供数据支撑

- Marvel：广告业务数据存储系统
 - 主要包括统一发布服务，数据存储服务，统一访问服务三个项目
- Marvel存储系统为广告核心模块提供数据支撑
- 在线存储量：1P+



业务场景核心需求：实时响应，存储量大，快速批量更新

- 实时响应

- Marvel为在线广告服务提供查询，访问延迟
直接关系广告曝光的成功率
- 99.9%响应延迟<10ms

- 海量存储

- 在线数据量100T+，三地六副本数据总存储
量1P+

- 快速批量发布

- 单表数据源100+，平均每天进行近百次全量
更新
- 大部分都是全量更新，即每次更新都需要把
所有key的数据都更新一遍
- 1T数据1小时以内发布上线

	数据量	ID数量	日更新量	日更新条数	日访问量
用户画像	10T+	百亿级	10T+	千亿级	千亿级
用户特征	10T+	百亿级	10T+	千亿级	千亿级
DMP数据	10T+	百亿级	1T+	千亿级	千亿级
新鲜度数据	10T+	数十亿	1T+	百亿级	千亿级

解决方案概览：腾讯广告自研数据存储系统（代号Marvel）

- Marvel是腾讯广告自研数据存储系统，立足广告业务场景核心需求：存储量大，数据更新量大，实时响应。
 - 低存储成本：数百台磁盘型服务器*，折合年成本近2000万
 - 快速批量更新：日发布近百批次，数十T，1小时内更新生效
 - 实时响应：99.9%延迟5ms

选型	存储/成本	更新	99.9%访问延迟
CKV CMEM	15000万	小时级别更新	5ms
CKV CMEM + TSSD	<1亿	不可控	10ms
HBase	5000万	天更新*	200ms
Marvel	2000万	1小时更新	5ms

* 800台TS80A：现网规模Marvel存储服务器规模，部分集群还未升级到最新版本，升级以后可以再节省部分机器

* 天更新：HBase索引合并严重影响在线服务性能，因此规划在晚上进行索引合并，导致每天只能进行一次发布

目录

1 Marvel存储系统简介

1.1 Marvel简介 / 1.2 核心业务场景 / 1.3 核心能力

2 Marvel批量发布架构演进

2.1 基于内存KV架构：满足实时响应需求 / 2.2 HBase-Extended Marvel架构：解决成本问题

2.3 HBase-Free Marvel架构：解决发布速度问题 / 2.4 HBase-Free Marvel架构分阶段实施

3 Marvel流式发布架构演进

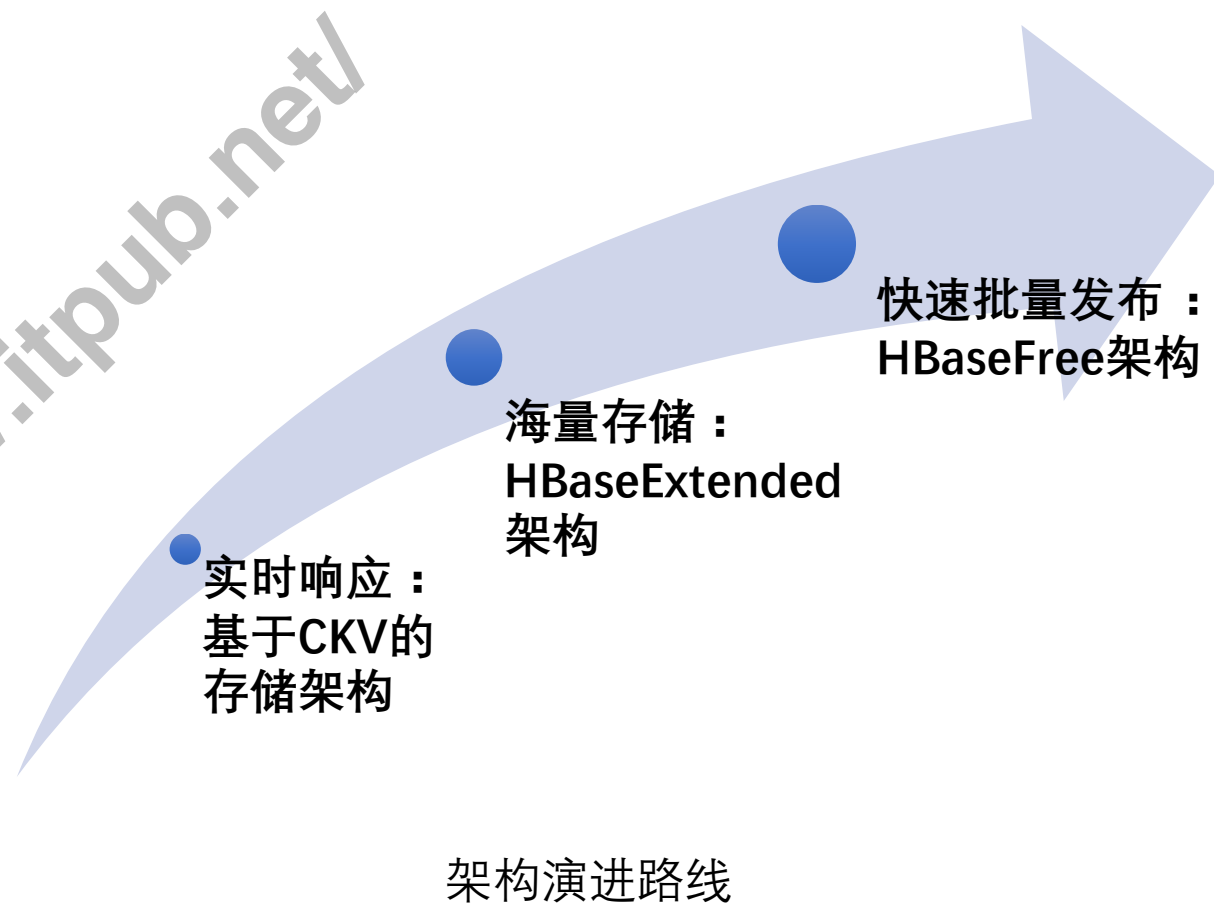
3.1 HBase-Extended架构：满足流式数据发布需求 / 3.2 HBase-Free架构：解决数据一致性问题

3.3 HBase-Free架构分阶段实施

4 总结

Marvel批量发布架构演进概述

- 批量发布：
 - 数据通过批量加载的模式发布，对发布数据吞吐量要求较高，时效性要求较低。发布周期一般为天或小时级别
- 核心业务需求：实时响应，海量存储，快速批量发布
 - 实时响应：核心需求，直接关系线上服务的质量，影响线上收入
 - 海量存储：只影响成本，对业务需求没有本质影响。另外，考虑到项目早期数据量相对较少，因此可以暂时不做重点考虑
 - 快速批量发布：项目早期，做到天级别更新可以满足基本需求
- 架构演进路线



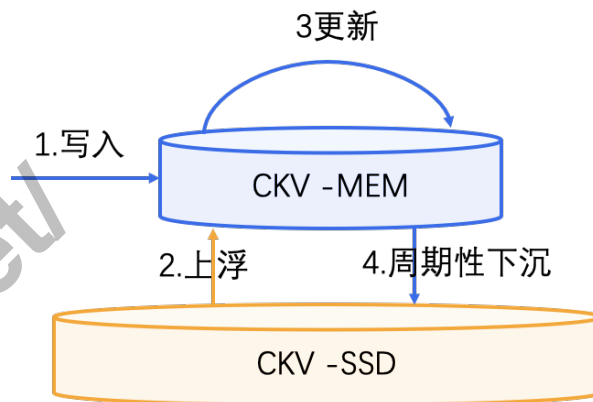
基于CKV的存储架构：解决实时响应的问题

- CKV是腾讯自研的内存型分布式KV系统，支持将部分冷数据存储在SSD磁盘中

- 解决问题：实时响应问题
- 潜在问题：CKV冷数据上浮问题：数据写入需要将数据从磁盘加载到内存，内存不足时会导致停写。

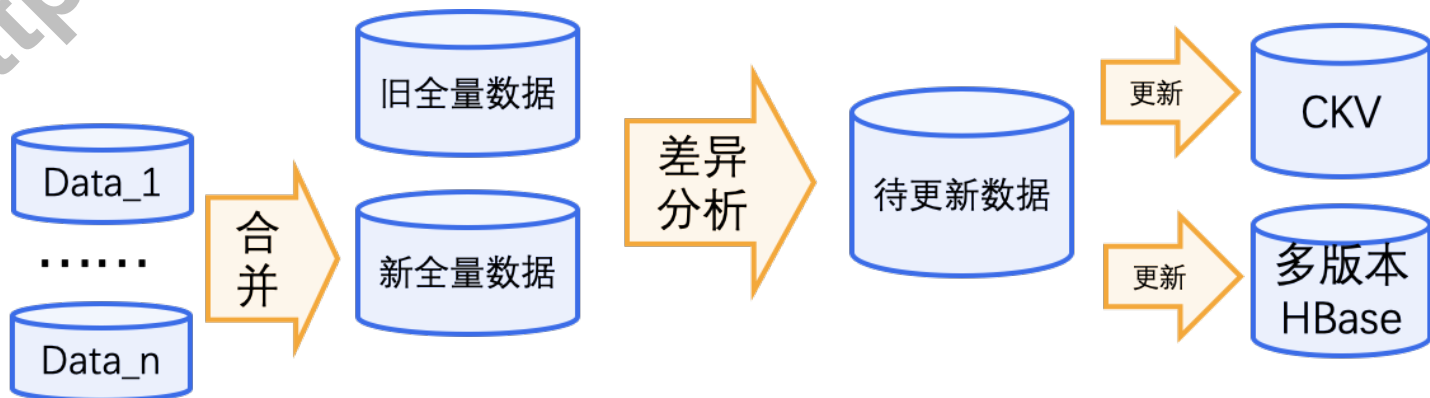
- 冷数据上浮问题解决方案：

- 冲突：减少内存量-》减少更新key的数量
- 全量更新：每个key都要更新
- 解决方案：离线做差异分析，线上只更新修改的数据



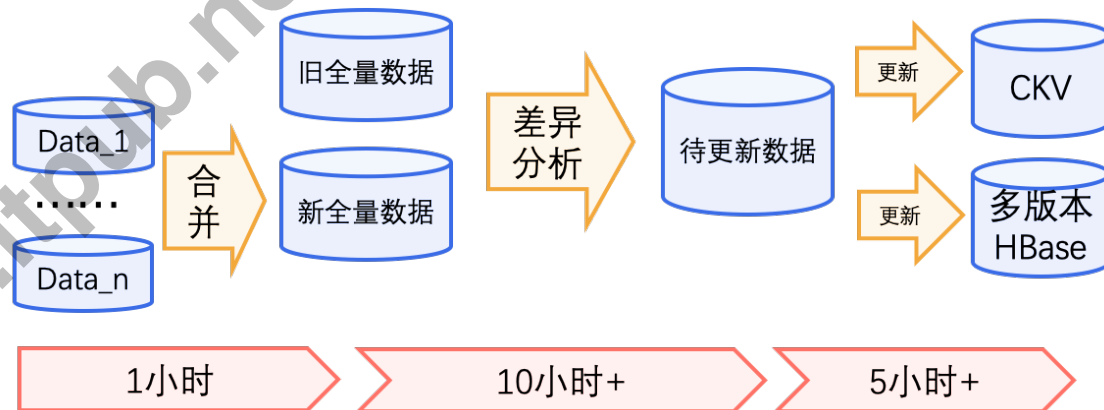
CKV数据更新流程

基于CKV架构数据更新流程



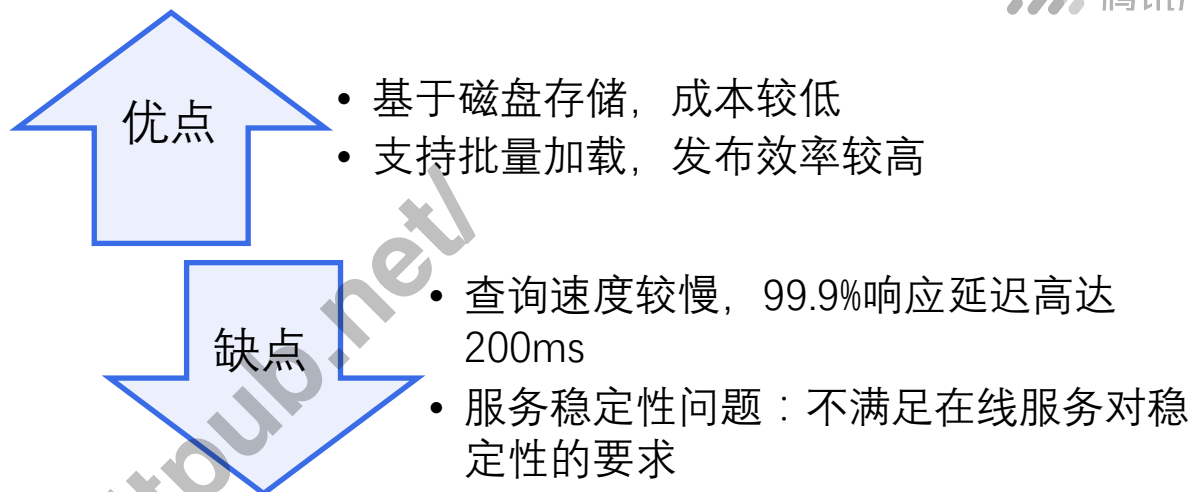
基于CKV存储架构问题及其改进方向

- 存储成本高
 - 数据存储需要大量内存
 - 数据更新需要先更新到内存
- 更新不受控性问题
 - 每天需要写入大量kv，内存不够会导致停写
- 发布周期长
 - 需要合并数据，并跟历史数据做差异分析
 - CKV不支持批量加载，只能点更新

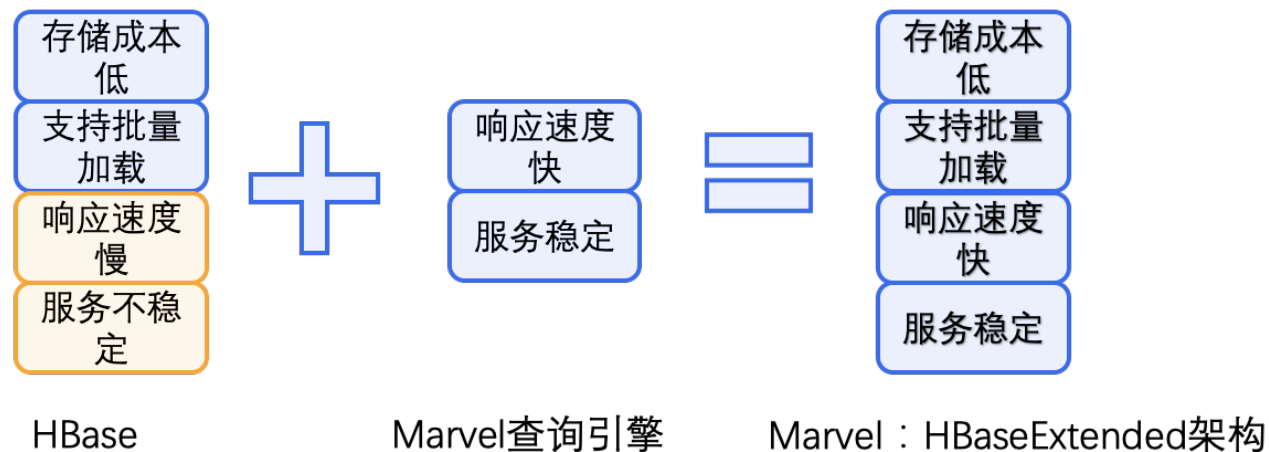


批量发布架构：架构升级选型

- 目标：解决CKV成本高，更新不受控问题
- 架构选型：
 - 解决成本高问题：基于磁盘的存储系统，首选HBase
 - HBase存在响应速度较慢，服务稳定性不足等问题
- 应对方案
 - HBase+Marvel，HBase解决存储/更新问题，Marvel解决查询速度和稳定性问题

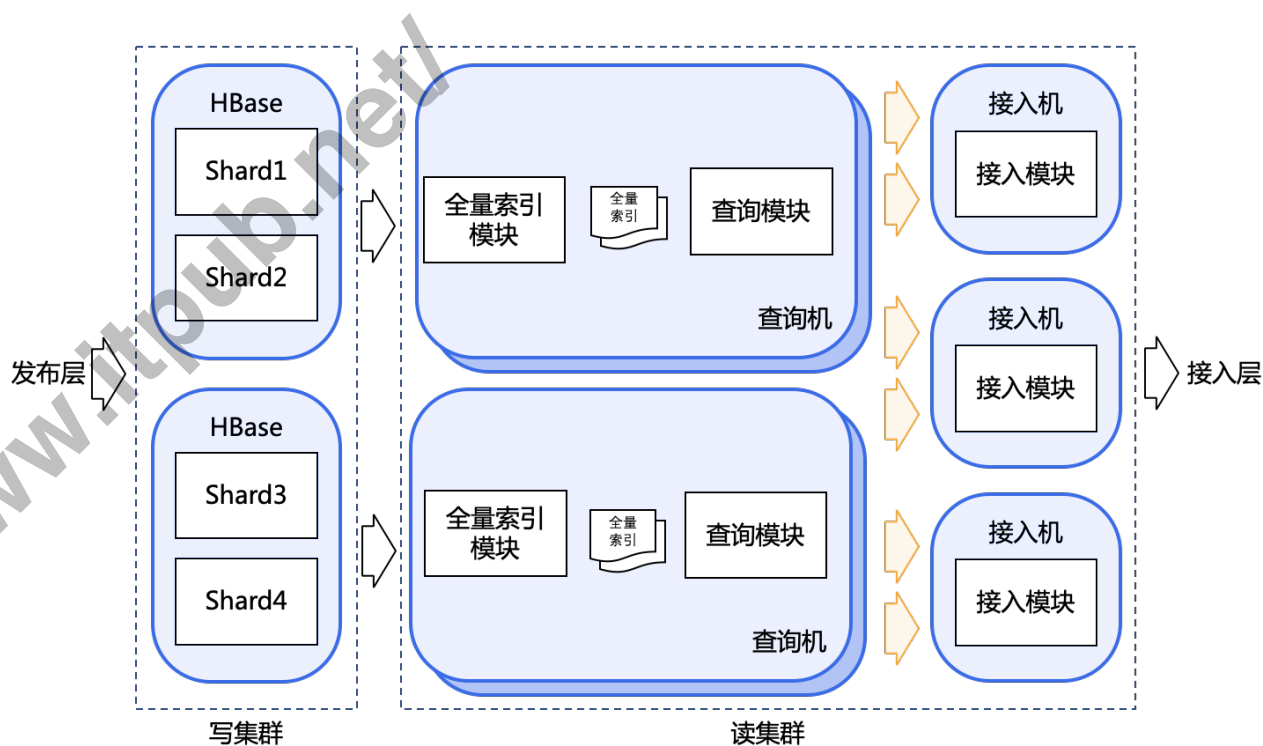


HBase优缺点分析



批量发布架构：HBase-Extended架构

- HBase作为存储后台 + 自研查询模块
- 写集群
 - 由HBase组成，接收发布系统发布的数据，并提供给索引模块
- 读集群
 - 全量索引模块：负责从HBase拉取数据，并转换成Marvel索引，供查询模块查询
 - 查询模块：负责从索引里面检索数据
 - 接入模块：负责接收请求，并将请求路由到合适的查询模块



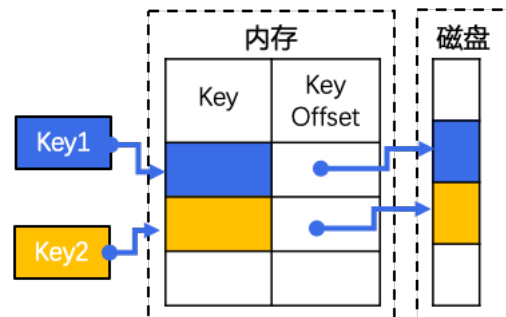
- HBase海量存储：HBase数据存储在磁盘上，成本较低
- HBase批量发布：通过Spark集群直接在HDFS生成HBase索引，通过API通知HBase直接加载（即HBase Bulk-Loading能力）

批量发布架构：HBase-Extended架构：单文件全内存索引，直接索引到Key 腾讯广告

- Marvel索引：

- 单文件存储：只保存一份全量索引文件
- 全内存索引：索引全部预先加载到内存，内存直接检索到key对应的数据位置

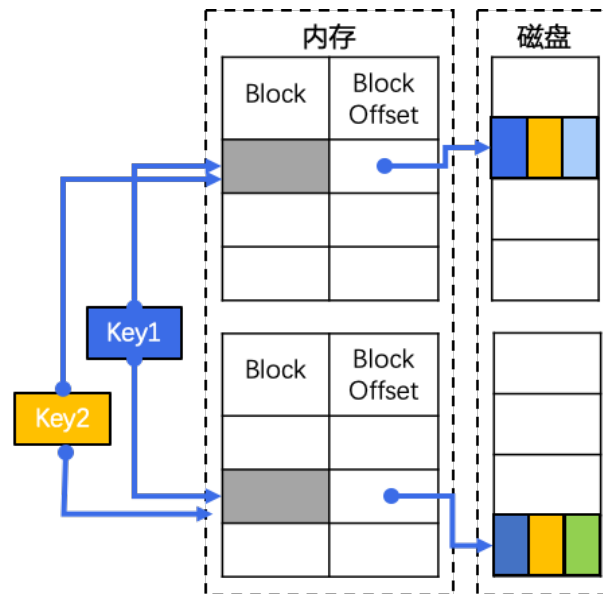
Marvel索引：单文件全内存索引



- HBase索引 (SSTable)：多文件存储 + 两级索引

- 多个文件存储：数据存在多个文件中，多次查询再合并
- 一级索引（内存）：数据块索引
- 二级索引（磁盘）：数据块内有序存储，顺序扫描*块内数据，查找指定Key

HBase索引：多文件两级索引



- 顺序扫描：在LevelDB等改进版实现中，先二分查找到基准位置，再顺序扫描
- 处理Value个数：查找时，HBase需要将整个数据块读取/解压以后再开始扫描

批量发布架构：HBase-Extended架构：单文件全内存索引，直接索引到Key

腾讯广告

• 单次查询性能对比

- 理论分析对比（参数设置：Value大小 = 1KB，HBase数据块大小 = 64KB，HBase索引文件数 = 4）
- 实际测试*对比：Marvel索引 VS. HBase索引（C++版本）

	Marvel	HBase
IO次数	1	4
处理Value个数*	1	256

理论分析性能对比

	Marvel索引	HBase索引
99.9%响应速度	1.825ms	5.198ms
CPU使用率	400%	1200%

实际测试性能表现

- 实际测试：测试60000QPS下两个索引的响应速度和CPU使用率，其中HBase索引也采用单文件存储模式；线上HBase环境采用多文件存储模式，性能差距会更大

批量发布架构：HBase-Extended架构：磁盘读写分离，双路竞速

- 磁盘读写分离

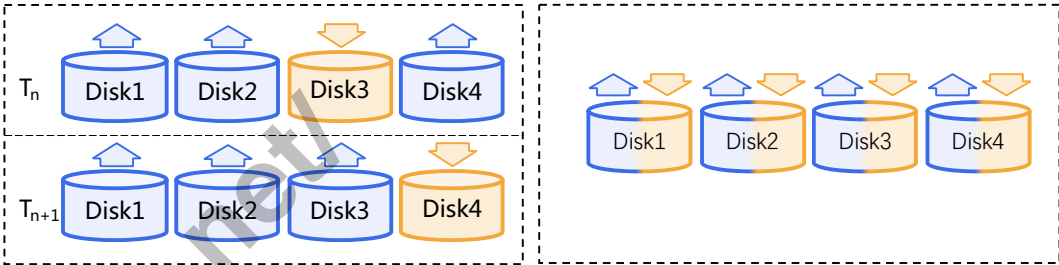
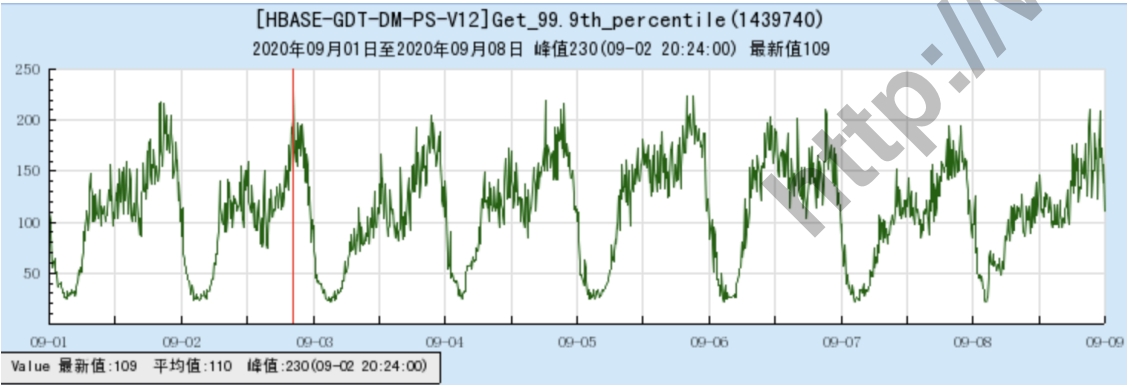
- 轮转换盘：多个盘，只有一个盘在建下一轮的全量索引，其他盘提供只读查询

- 双路竞速

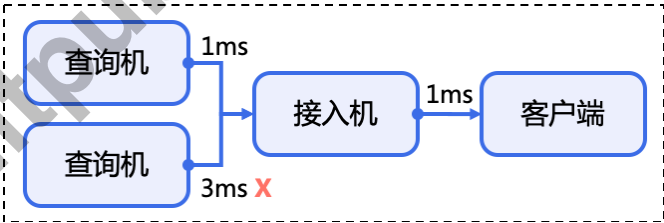
- 同时请求两个查询机，取较快的结果返回给客户端
- 前提：查询性能高，双路查询对成本影响可控

- 读性能对比，99.9%读延迟：

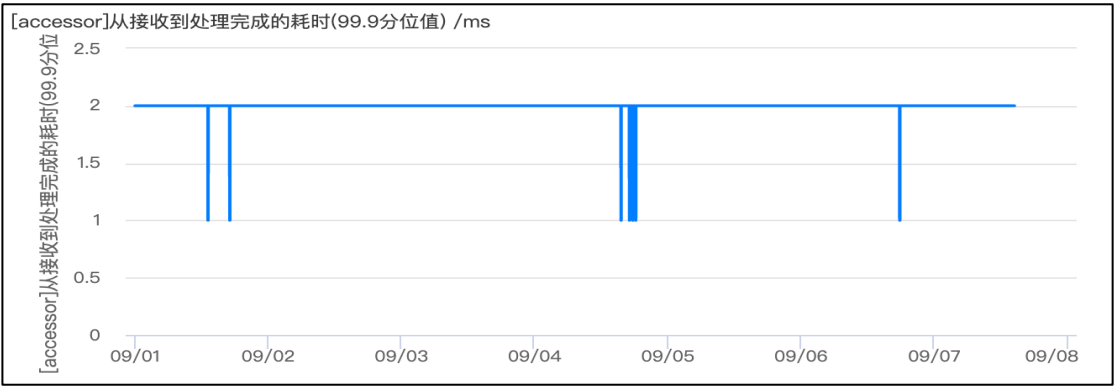
- HBase：百毫秒级别* vs. Marvel：1-2ms



Marvel读写分离 vs. HBase读写混合



双路竞速提升平均响应速度1倍，999响应速度3-5倍



Marvel内部延迟1-2ms

HBase 99.9%内部延迟 200ms

* 百毫秒级别：监控来自相同磁盘配置HBase集群，线上系统99.9%性能表现，99p:50ms左右，95p:2-10ms

HBase-Extended：总结：高性能/实时响应，解决HBase查询能力问题



- 对HBase的功能做了扩展，实现了高性能，实时响应的查询引擎，性能是HBase的10倍+

- 单机QPS*：15万/秒
- 99.9%响应速度：<5ms

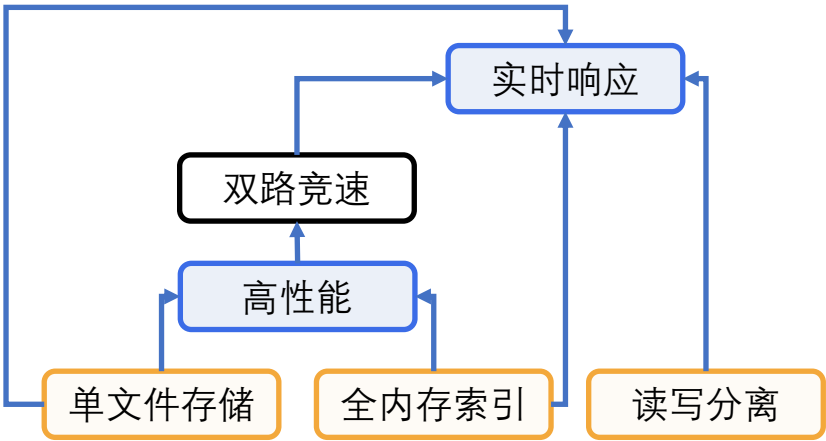
- Marvel高性能/实时响应的三大基石

- 单文件存储
- 全内存索引
- 读写分离

- HBase-Extended架构充分利用了HBase和Marvel各自的优点

- HBase：海量存储/批量加载能力
- Marvel：高性能实时查询的能力

	HBase	Marvel
单机峰值QPS	1.5万/秒*	15万/秒
999响应延时	200ms	<5ms



Marvel高性能三大基石

单机QPS：Marvel线下压测中（除单独注明外，这里提到的机器都是TS80A），单机QPS可达30万/s，线上环境中，为了确保网络稳定，只用30%-50%万兆网卡带宽，实际QPS不超过15万/秒

1.5万/秒：HBase线下压测时，BulkLoad一次后Compaction的过程中，QPS是1.88万，每增加一次BulkLoad性能降低20%左右，生产环境机器一般预算QPS 1万/秒

批量发布架构演进：HBase-Extended架构问题描述



• 数据发布时延问题

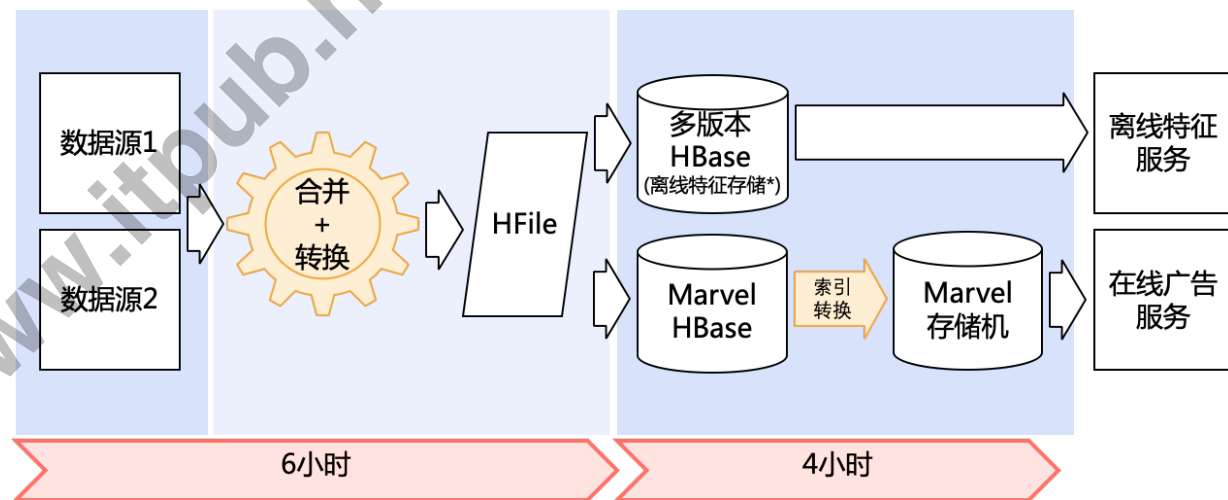
- HBase-Extended架构下，数据发布耗时过长，数据发布周期只能做到**天更新**，单次发布耗时10小时。

• 数据发布流程

- 每天将索引数据合并，转换为HBase索引，然后将HBase索引批量发布（BulkLoad）到多版本HBase和Marvel HBase（6小时）
- Marvel定期从HBase拉取全量数据，转换为Marvel索引（4小时）

• 发布时延过长的影响

- 影响研发迭代效率：从实验数据发布到数据生效，需要一天以上的时间
- 影响系统安全：问题数据回滚需要一天
- 影响数据价值：数据时效性得不到保障



* 离线特征存储：由于用户特征数据量比较大，在线服务不会把特征数据落到log中，只会在log中写入特征时间戳；离线训练模型时需要用时间戳从多版本历史数据中查找具体的特征值。

HBase-Extended架构问题分析：延迟瓶颈分析

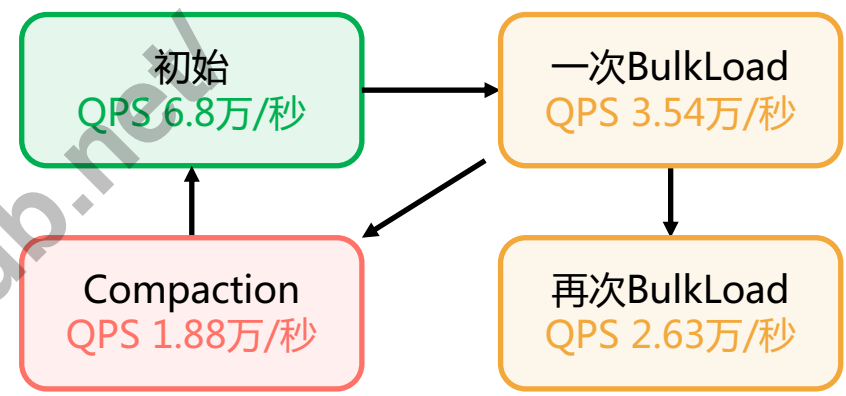


- HBase BulkLoad 能力有限，一天一次

- BulkLoad会导致索引文件数量增加，影响性能
- 文件数量需要通过索引合并(Compaction)来减少
- Compaction对查询性能带来很大影响，故安排在夜间进行

- Marvel定期从HBase拉取全量索引，耗时4小时

- 控制索引转换对SSD磁盘寿命的影响
 - 每天写6遍，以磁盘利用率40%算，折合2.4倍SSD容量。
- 控制索引转换的性能消耗
 - 解压缩 + 格式转换 + 压缩，单线程大约5MB/秒
 - 16线程建索引，1T数据大约耗时3.6个小时



BulkLoad/Compaction对性能的影响

厂商	型号	容量 (TB)	DWPD*	每天写入量 (TB)
Huawei	HWE52P432T0L002N	1.8	2	3.6
Huawei	HWE32P43020M000N	2	3	6
Samsung	MZWLK1T8HCHP	1.8	3	5.4
Intel	SSDPE2KE020T7	2	3	6
Intel	SSDPE2ME020T4	1.8	3	5.4
Intel	SSDPE2ME020T4	2	3	6

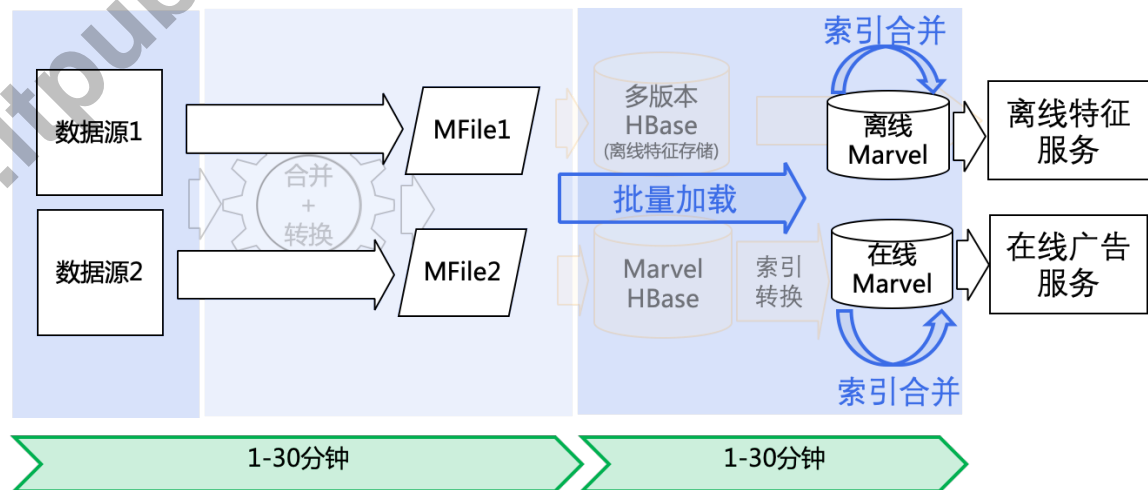
* DWPD , Drive Writes Per Day , 每天允许写入次数

腾讯NVME磁盘及每天允许写入量

HBase-Free架构概述：解决方案-去HBase



- 解决方案：将原来由HBase承担的功能，都合并到Marvel
 - 解决HBase不能频繁批量加载的问题
 - 不需要等待Marvel索引转换：一旦数据批量发布到Marvel，数据就可以马上对外提供查询
- 新发布流程
 - 数据源到达时，直接离线转换为Marvel索引；
 - Marvel直接加载Marvel 索引，并对外提供查询服务
 - 在提供查询服务的同时，后台启动索引合并任务，将小索引文件合并成大索引问题



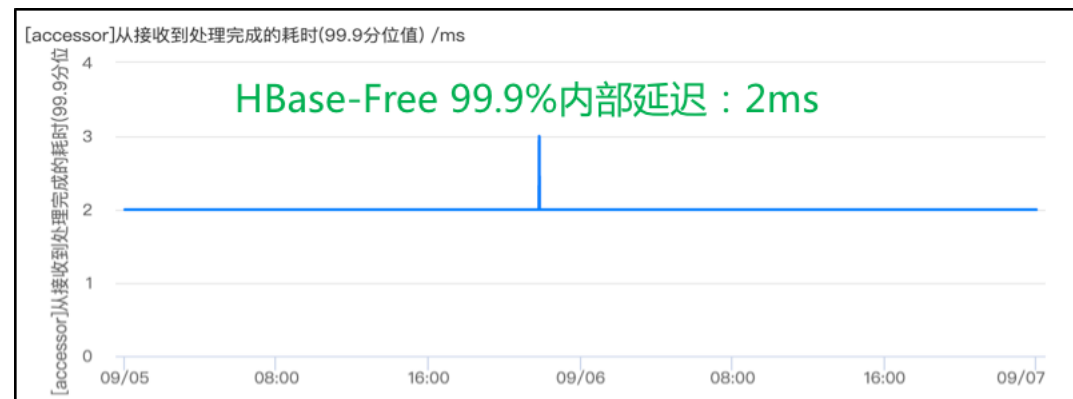
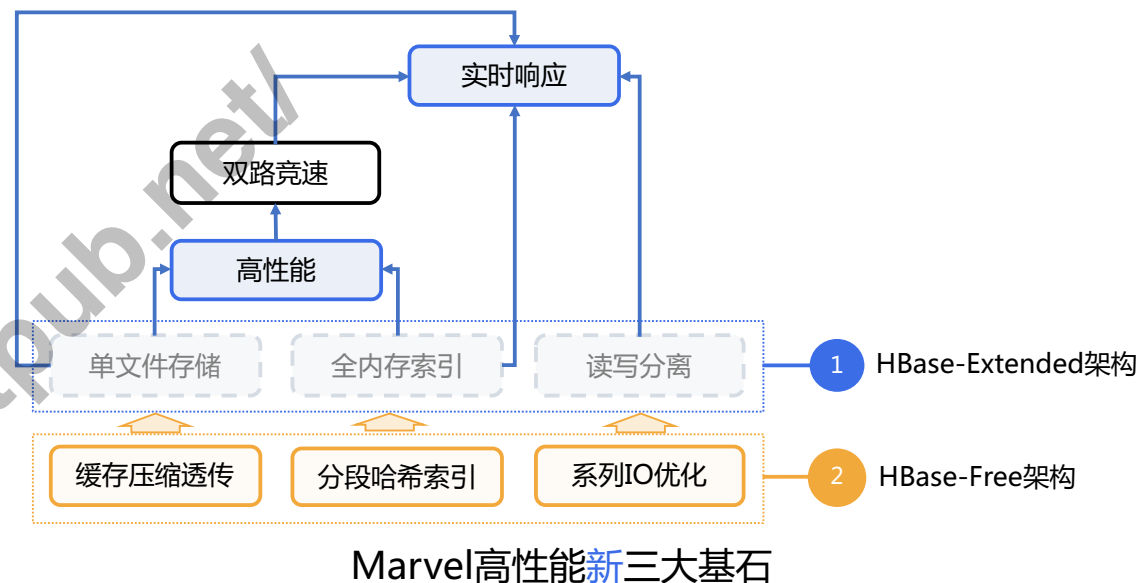
不能读写分离：新架构需要不断加载新发布的文件，导致索引文件增多，影响性能，为了提高性能需要在后台不断合并小文件，从而导致读写控制更加复杂，无法做到读写分离

HBase-Free架构工程优化：解决架构升级带来的性能问题和响应速度问题

腾讯广告



- 性能问题：查询多个文件，合并多个结果，导致性能下降
 - 解决方案：引入缓存压缩透传方案，解决性能问题
- 内存问题：同时加载多份索引，索引占用内存增加
 - 解决方案：改进索引结构，压缩索引项大小到5byte以下
- IO问题：不断加载新索引，合并索引数据，导致IO负载变重，响应速度变慢
 - 解决方案：通过Linux Native AIO，Direct IO，写盘限速，并行读取等系列IO优化，解决IO竞争问题

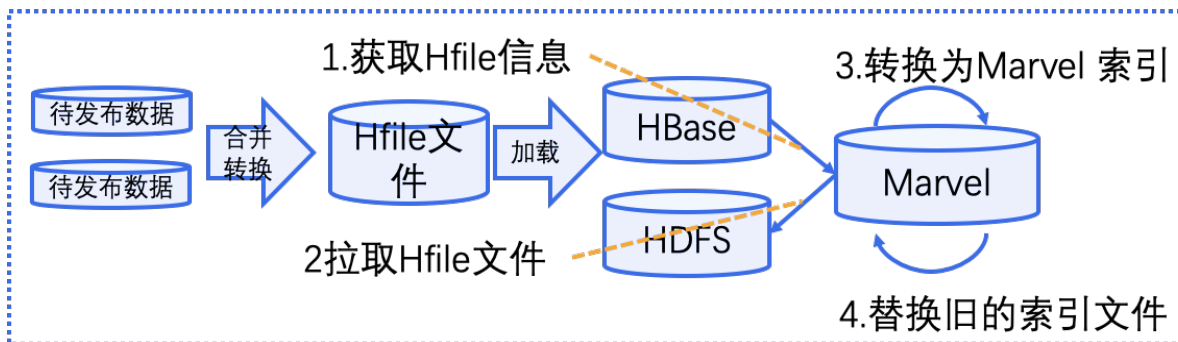


Marvel：HBase-Free架构内部延迟

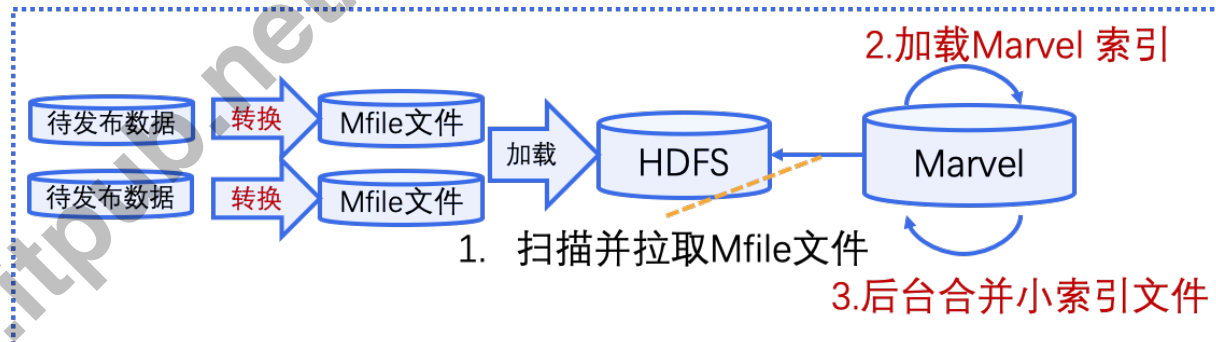
Marvel : HBase-Free架构优化分阶段实施



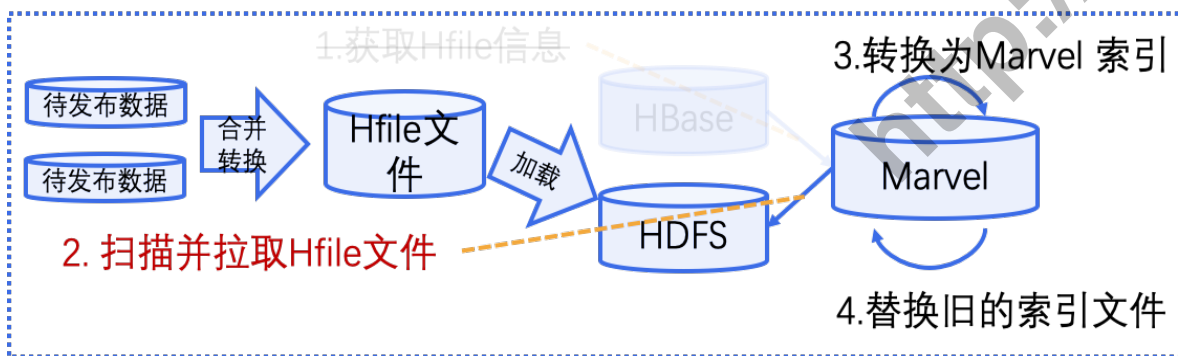
- 考虑到线上服务的稳定性，我们对架构优化做了分阶段实施



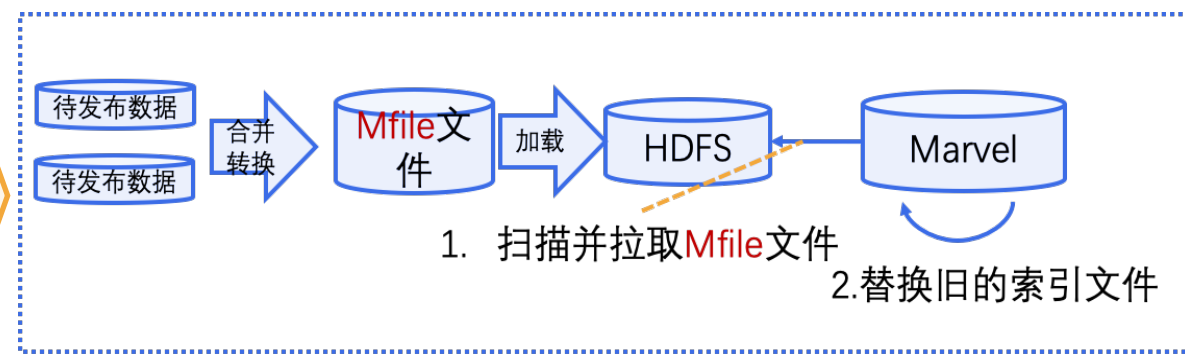
原始架构：依赖HBase



最终架构：直接发布Marvel文件，Marvel后台进行合并



改进1：去HBase，直接加载Hfile全量索引



改进2：直接生成Marvel全量索引

目录

1 Marvel存储系统简介

1.1 Marvel简介 / 1.2 核心业务场景 / 1.3 核心能力

2 Marvel批量发布架构演进

2.1 基于内存KV架构：满足实时响应需求 / 2.2 HBase-Extended Marvel架构：解决成本问题

2.3 HBase-Free Marvel架构：解决发布速度问题 / 2.4 HBase-Free Marvel架构分阶段实施

3 Marvel流式发布架构演进

3.1 HBase-Extended架构：满足流式数据发布需求 / 3.2 HBase-Free架构：解决数据一致性问题

3.3 HBase-Free架构分阶段实施

4 总结

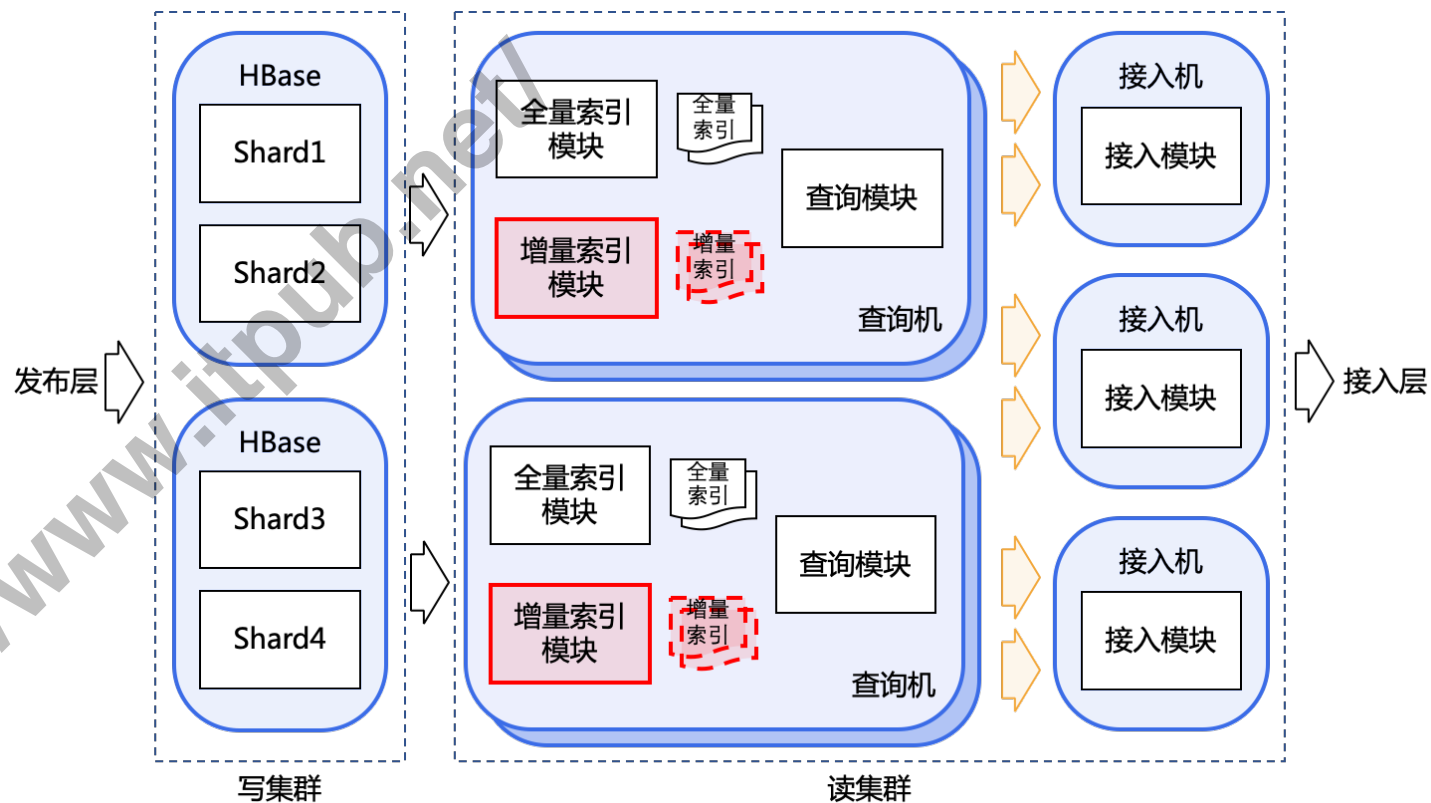
流式发布架构演进概述

- 流式发布架构：数据通过流式系统（spark/flink/storm）写入，对数据生效时效性要求比较高，要求写入以后短时间内数据就可以被查询到
- 流式发布业务需求变化：
 - 最终一致性：开始阶段，Marvel管理的数据基本上画像/特征/策略数据，这些数据对数据一致性的要求比较宽松，只要求最终一致性
 - 强一致性：由于Marvel相对HBase有很大的性能和稳定性优势，因此更多使用HBase的业务场景也希望迁移到Marvel。这部分应用场景要求数据强一致性

- HBase海量存储：HBase数据存储在磁盘上，成本较低
- HBase批量发布：通过Spark集群直接在HDFS生成HBase索引，通过API通知HBase直接加载（即HBase Bulk-Loading能力）

Marvel流式发布架构：HBase-Extended架构：最终一致性

- HBase作为存储后台 + 自研查询模块
- 写集群
 - 由HBase组成，接收发布系统发布的数据，并提供给索引模块
- 读集群
 - 全量索引模块：负责从HBase拉取数据，并转换成Marvel索引，供查询模块查询
 - 增量索引模块，从HBase拉取增量流水，并转换成Marvel索引，供查询模块查询
 - 查询模块：负责从索引里面检索数据
 - 接入模块：负责接收请求，并将请求路由到合适的查询模块



- HBase海量存储：HBase数据存储在磁盘上，成本较低
- HBase批量发布：通过Spark集群直接在HDFS生成HBase索引，通过API通知HBase直接加载（即HBase Bulk-Loading能力）

流式发布架构：HBase-Extended架构：数据发布流程

• 数据发布流程

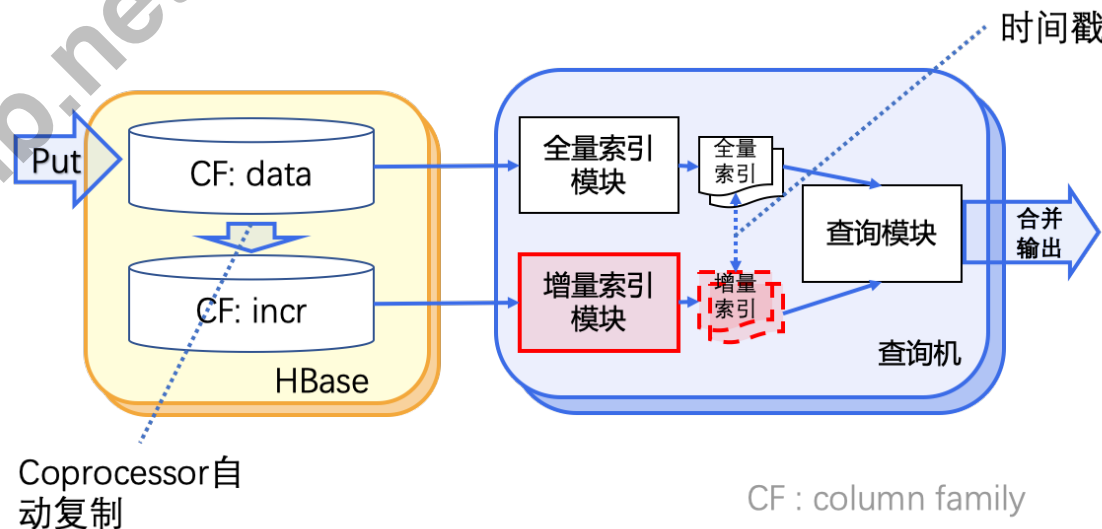
- 数据通过HBase写入接口写入HBase
- 数据写入的同时，HBase通过coprocessor生成修改流水
- Marvel增量模块通过拉取修改流水获取最新改动
- Marvel拉取下一次全量以后，增量索引模块会丢弃过期增量数据

• 数据查询流程

- 查询模块分别从全量索引和增量索引中查询数据，合并后的到结果

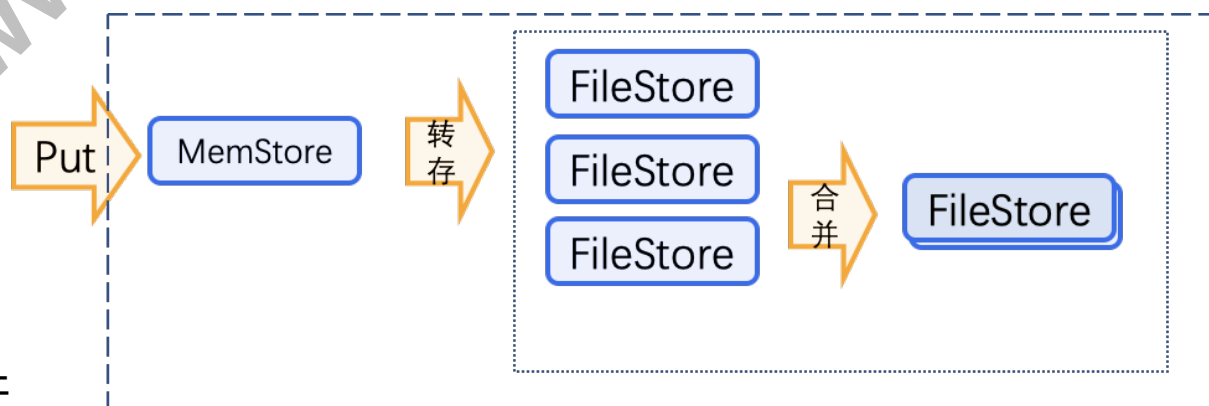
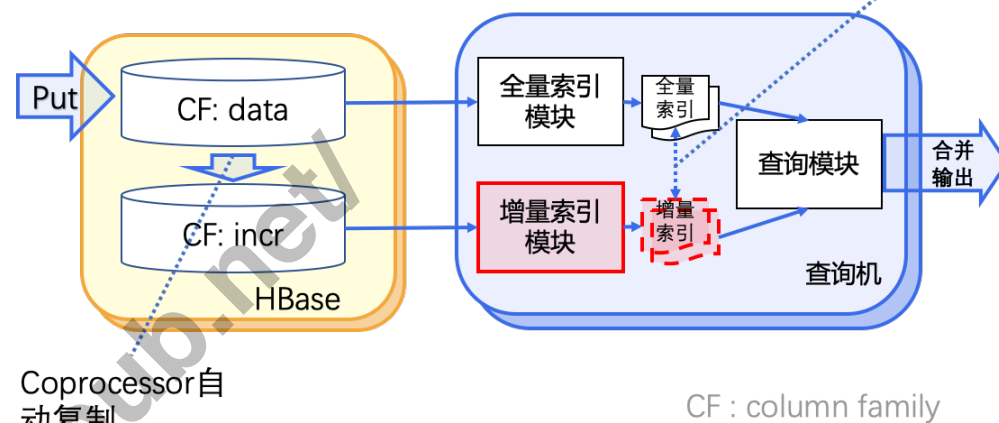
• 架构要点

- 在批量发布的基础上，增加了增量模块
- HBase：负责数据的可靠性
- Marvel：高性能查询



流式发布架构：HBase-Free：强一致性

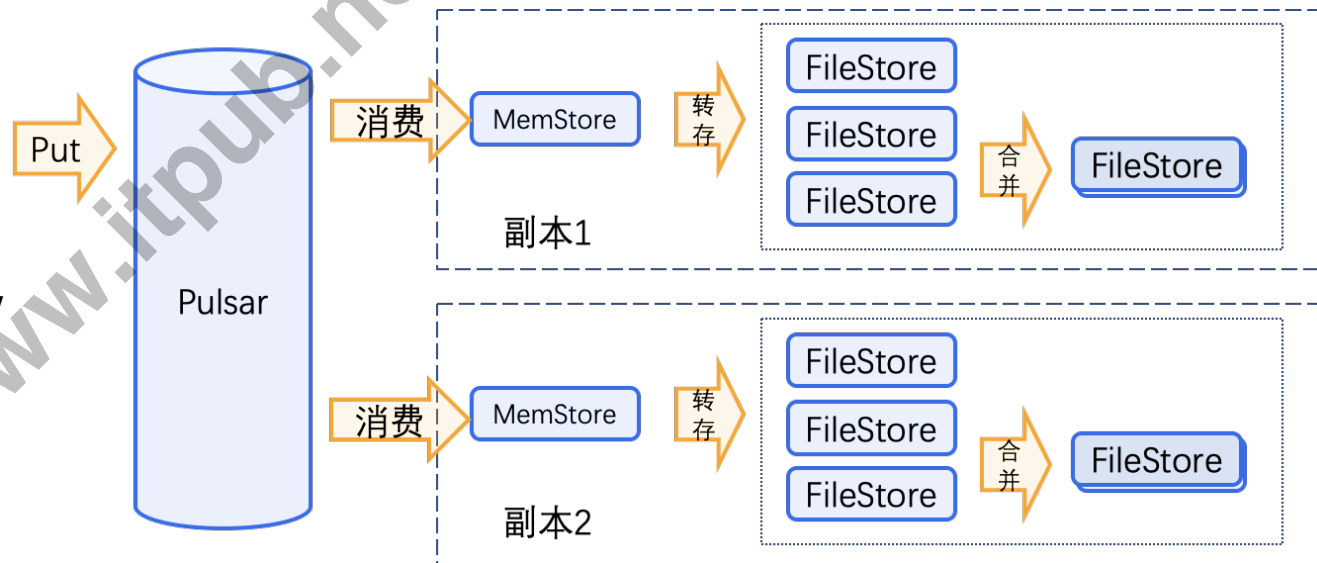
- 缺点：
 - 只能提供最终一致性，写入数据不能立即可见。
 - 引入HBase依赖，HBase相对比较复杂。
- 需求：提供写入功能，保证数据强一致性
 - 业务需求：提供强一致性写入支持
 - 架构简化需求：去除对HBase依赖
- 架构改进：参考HBase架构
 - 新的数据修改先写入内存
 - 内存块满了，转存成磁盘索引文件
 - 后台启动任务将小索引文件合并成大索引文件



- HBase海量存储：HBase数据存储在磁盘上，成本较低
- HBase批量发布：通过Spark集群直接在HDFS生成HBase索引，通过API通知HBase直接加载（即HBase Bulk-Loading能力）

流式发布架构：问题：多副本一致性及解决

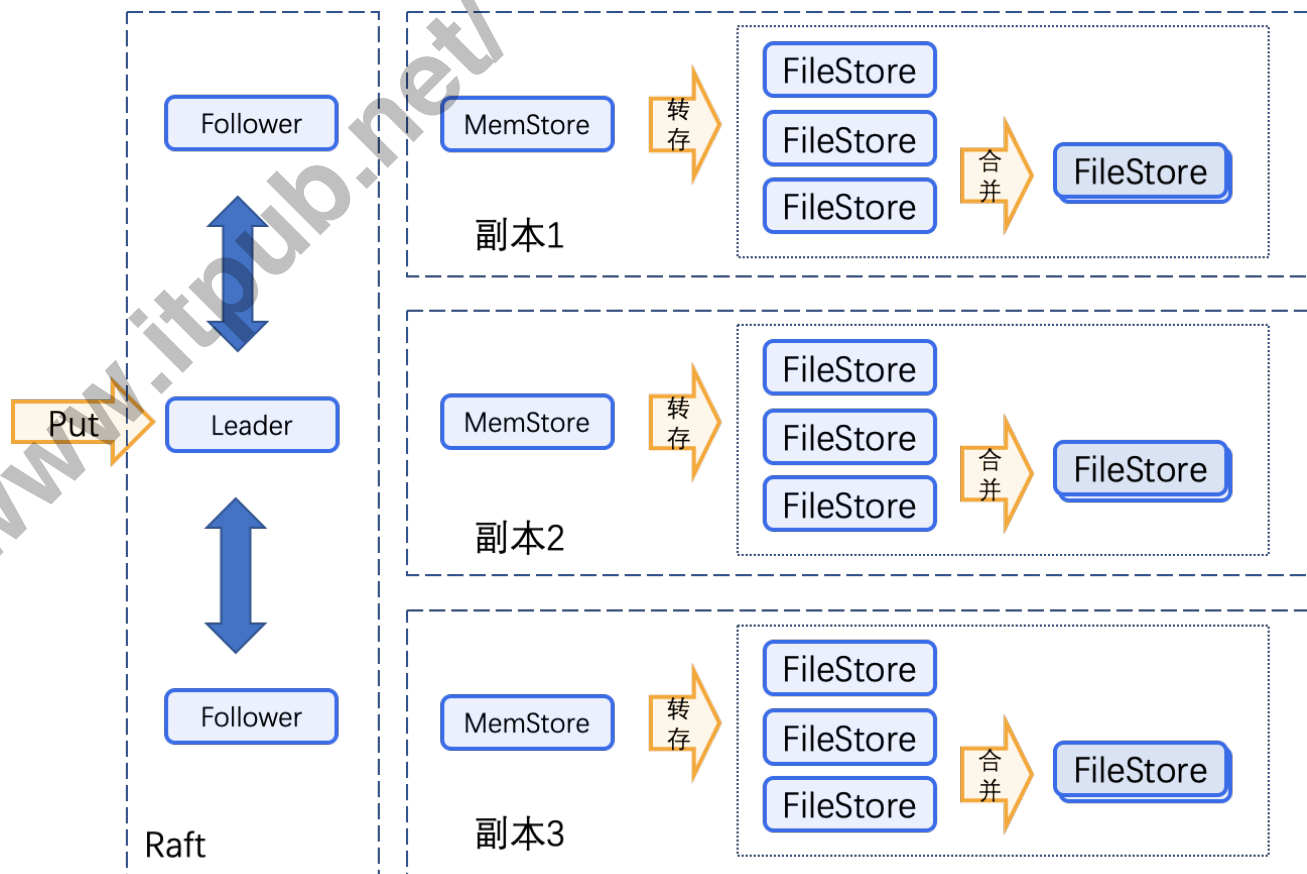
- 缺点：
 - 只能保证单机正常运行，无法保证多副本一致性
- 解决方案：解决多副本一致性问题 & 冷启动问题
 - Raft？缺点：过于复杂，架构升级幅度过大，影响线上安全
 - 消息队列接收写入数据，并同步到各个副本
- 新架构：
 - Pulsar接收写入，各副本同时消费Pulsar实现最终一致性



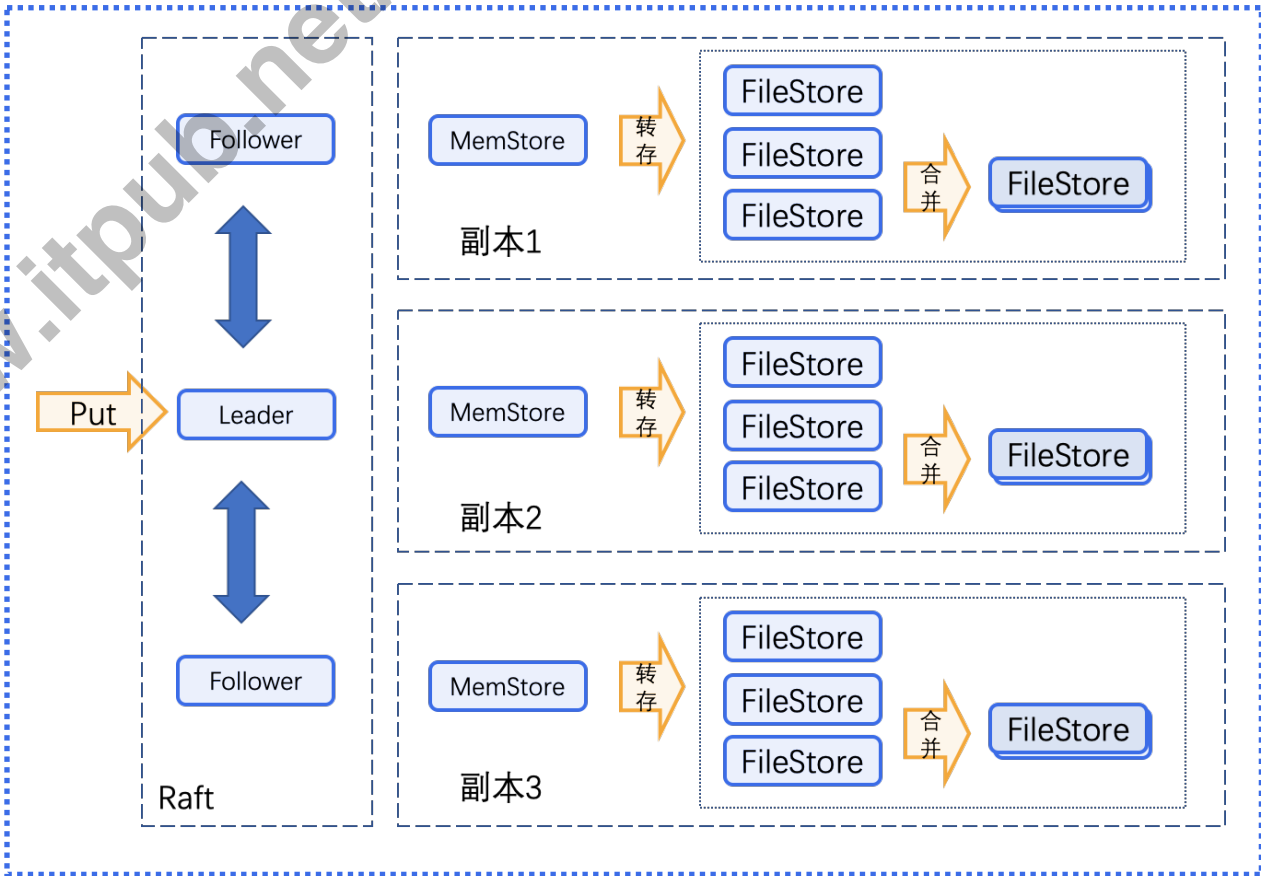
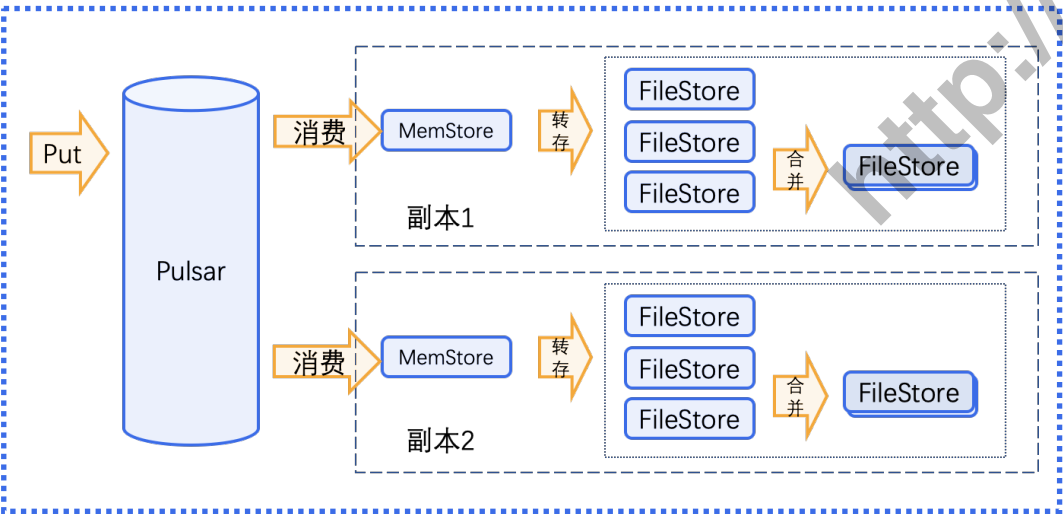
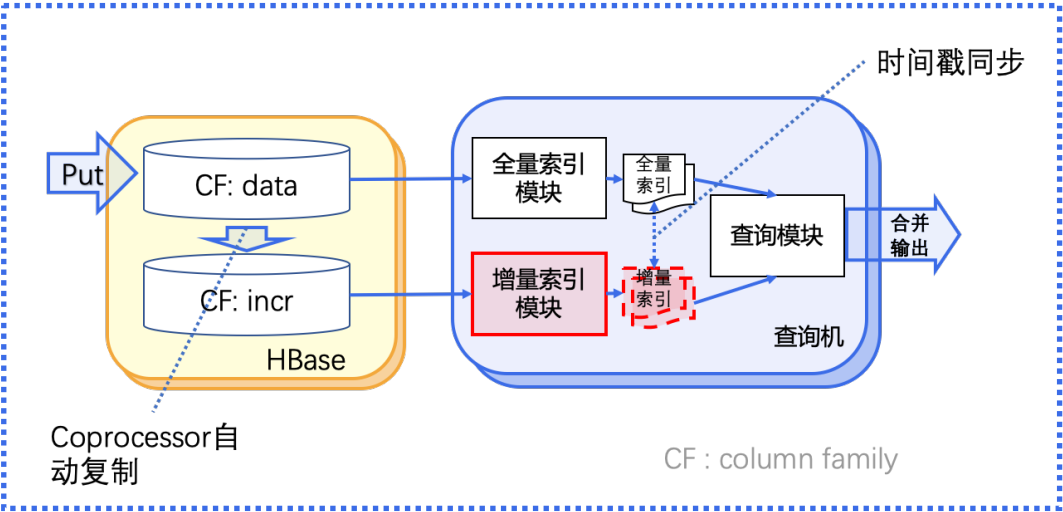
- HBase海量存储：HBase数据存储在磁盘上，成本较低
- HBase批量发布：通过Spark集群直接在HDFS生成HBase索引，通过API通知HBase直接加载（即HBase Bulk-Loading能力）

流式发布架构：实现强一致性

- 原架构缺点：使用消息队列实现数据一致性，只能实现最终一致性，无法实现写入立即可见
- 解决方案：解决多副本一致性问题
 - Raft：实现数据同步
- 提供多种一致性级别：
 - 强一致性：只从leader 读取数据
 - 最终一致性：只从follower读取或者同时从leader/follower读取



流式发布架构演进



目录

1 广告业务数据存储系统简介

1.1 模块定位 / 1.2 现有存储系统问题 / 1.3 解决方案概述

2 Marvel批量发布架构演进

2.1 基于内存KV架构：满足实时响应需求 / 2.2 HBase-Extended Marvel架构：解决成本问题

2.3 HBase-Free Marvel架构：解决发布速度问题

3 Marvel流式发布架构演进

3.1 HBase-Extended架构：满足流式数据发布需求 / 3.2 HBase-Free架构：解决数据一致性问题

3.3 HBase-Free架构分阶段实施

4 总结

- 架构选型/演进方向
 - 优先满足核心需求
 - 优先考虑成熟基础设施，必要时逐步优化替换
 - 核心需求满足并稳定运行以后，再考虑如成本，时效性之类重要需求
- 架构演进实施
 - 对于一个承载线上重要业务的系统，需要分阶段进行迭代升级
 - 架构演进指导思想：安全第一，业务优先，兼顾性能/架构优化



扫描二维码，了解腾讯广告更多技术干货

Thanks

<http://www.itpub.net/>

